



← Back to student guides

 DVC

Data versioning

3 levels

113 sections



The Complete Guide

Taught at three levels, written three ways. Pick Beginner, Mid-level, or Senior, then read the full explanation, a fast interview review, or the practical tips and traps for that level.

CODE

POINTER

CACHE

REMOTE



Quick Start

 Open PDF

A one-page starter for this tool. Scroll through it here, or open it full screen.



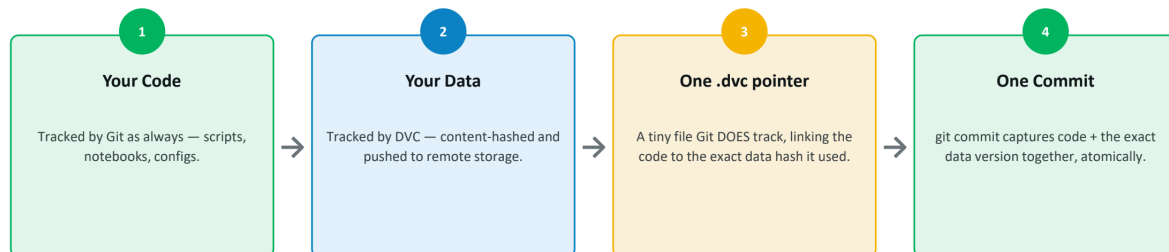


... and leaves behind a tiny pointer file
in Git that holds only a hash.

Git for code. DVC for data. Same commands, same habits, one workflow.

THE PROBLEM → THE FIX

How Git and DVC Divide the Work



01

Versioning Large Data with DVC



Part three of three, and the one to read if you only read one. A cumulative review of **the entire series** (foundations, pipeline machinery, and the governance and scale work a senior owns) organised by topic rather than by level. About fifty minutes. Fast review first, common questions at the end.

Part one: foundations



DVC versions data and ML pipelines on top of Git. A small hash-bearing pointer goes into Git; the bytes go into object storage. `dvc.lock` records every input and output hash from the last successful run, which is how `dvc repro` reruns only the stages whose inputs changed.

Why not Git alone: full history of every large binary, in every clone, forever. **Why not LFS:** it fixes size and nothing else: no pipelines, no graph, no metrics, no experiments, and storage tied to your Git host.

`git checkout` moves pointers; `dvc checkout` moves data. `dvc install` adds hooks so the second is automatic.

`dvc add` , in four steps

Hash the contents → move to the content-addressed cache → write the `.dvc` pointer → add a `.gitignore` entry. A clean `git status` afterwards is the signal it worked.

The commands and the Git parallel



<code>git add</code>	<code>dvc add</code>	working file → tracked
<code>git push</code>	<code>dvc push</code>	Cache → remote
<code>git pull</code>	<code>dvc pull</code>	Remote → cache → workspace
<code>git checkout</code>	<code>dvc checkout</code>	History → working files
<code>git status</code>	<code>dvc status</code> / <code>-c</code>	What differs, locally / remotely

The rules that never change

Never `dvc add` a pipeline output, because `outs` owns it. **Always commit** `dvc.lock`, never hand-edit it. **Declare every input in** `deps` or the stage silently skips. **Parameters go in** `params.yaml` and must be declared. **Metrics and plots use** `cache: false` so they diff from Git with no pull. **Credentials never go in** `.dvc/config`.

The traps

Symptom	Cause	Fix
Wrong data version after <code>git checkout</code>	Skipped <code>dvc checkout</code>	<code>dvc install</code> the hooks
Missing cache files on pull	Never pushed	<code>dvc push</code> where it was produced
<code>output 'x' is already tracked</code>	<code>dvc add</code> on a stage output	Remove the <code>.dvc</code>
Stage never reruns	Script not in <code>deps</code>	Add it
Parameter change ignored	Not declared in <code>params</code>	Declare it
<code>URL is not supported</code>	Missing driver	<code>pip install "dvc[s3]"</code>



The cache, precisely

```
.dvc/cache/files/md5/
```

Copy

```
|— 8f/2c4b19e0a7d3f1c6b8a2e4d7091f3b    ← file contents
|— 3a/7f9c2e8b1d4506...dir              ← JSON listing for a directory
```

MD5 by default: a content address chosen for speed, not a security control. Two-character shard directories. A `.dir` object maps relative paths to per-file hashes, so files inside a directory still deduplicate individually. **Hashing dominates, not transfer**; DVC caches hashes by `(path, size, mtime, inode)`, which is why a second `add` is near-instant.

Link types

Type	Extra disk	Safe to edit in place
<code>reflink</code>	None (CoW)	Yes
<code>hardlink</code> / <code>symlink</code>	None	No: silently corrupts cached history
<code>copy</code>	Double	Yes

Default `reflink,copy`; add `cache.protected true` if you use hardlinks so in-place edits fail loudly.

Templating and modifiers

```
YAML
```

Copy

```
stages:
  featurize:
    foreach: [tfidf, embeddings]      # matrix: for a cross product
    do:
```



push: false

regenerable; do not store 200 GB

Modifier	Effect
<code>cache: false</code>	Committed to Git: small, diffable text
<code>persist: true</code>	Not deleted before a rerun; the stage is no longer a pure function
<code>push: false</code>	Cached locally, never uploaded: the main cost lever
<code>remote: NAME</code>	Sent to a specific, usually colder, remote

External data, remotes, experiments

`dvc import-url` tracks an external source with `dvc update` ; `--no-download` records the pointer only; an external path in `deps` is hashed remotely. **A moving external path is a reproducibility hole**, so pin dated, immutable paths.

`jobs` is the biggest `pull` speed lever. `verify true` re-hashes on download.

`version_aware` uses natural paths plus bucket versioning: browsable, no cross-file dedup.

Queued experiments run in **isolated temporary workspaces**, which is what makes `--run-all --jobs 4` safe. They are Git refs under `refs/exps/` plus cached outputs, so thirty runs of a 2 GB model is 60 GB, cleared with `dvc exp remove` then `dvc gc --all-commits` .

DVCLive, and CI

DVCLive logs per step, turning endpoints into curves. In CI: OIDC for credentials, `fetch-depth: 0` for metrics diffs, `.dvc/cache` cached on `dvc.lock` , and `dvc status empty` as a **required check**, the gate that makes reproducibility enforceable. Push only from the protected branch.



The trust model

Actor	Needs	Must not have
Developer	Read; write on a dev prefix	Write on production data
CI on <code>main</code>	Read + write	Delete
CI on a fork PR	Read only	Any write
Retention job	Delete, scoped and audited	Write

Three properties make access sharper than it looks:

Content-addressed writes are silent overwrites. A client with write access can replace the object at a hash path, and every commit referencing that hash now resolves to wrong data with no error.

A pointer is not proof. Nothing verifies bytes on download unless `verify true` is set.

Deletion is the dangerous permission.

The single most dangerous command in DVC

`dvc gc --cloud --workspace` deletes every remote object not needed by the **currently checked-out** revision, which is catastrophic on a multi-branch repository, and it exits zero. Remove delete from human credentials, do retention with an audited lifecycle policy, and enable bucket versioning so a mistake is survivable.

Credentials with nothing stored

Mechanism	Stored secret	Scope
OIDC / workload identity	None	A specific repo, branch, or environment



Instance of pod identity	None	A machine or workload
Long-lived access key	Yes	Everything it allows, until rotated

The trust policy is where the security lives, so pin the `sub` to a branch. For humans, the provider's SSO; DVC uses each provider's standard credential chain. **A fork build must never hold a write credential.**

Immutability and retention

Control	Guarantees
Bucket versioning	An overwrite or delete is recoverable
Object lock, governance	Deletion needs a specific privilege
Object lock, compliance	Deletion is impossible until expiry, for anyone
Lifecycle to cold storage	Cheap long-term retention

Retention belongs in a **lifecycle policy** (declarative, audited, and not triggerable by a stale clone) not in `dvc gc --cloud`.

Cost

Charge	Driven by	Typical surprise
Storage	Bytes × time × class	Every experiment's model, forever
Requests	Object count	A million small files costs more in requests than bytes
Egress	Bytes leaving the region	CI in the wrong region
Retrieval	Restoring from cold	A "cheap archive" that is expensive to read



Scale, and where DVC stops

Shape	Do
100 × 10 GB files	Nothing. This is DVC's sweet spot
1M × 4 KB files	Shard into tars, Parquet, WebDataset, LMDB
Frequently appended table	Partition by date, track partitions
Warehouse table queried by SQL	Track the query and result hash; let Iceberg/Delta version the table

✓ DVC is right

- File-shaped data: images, audio, text, models
- Data versioned alongside the code that reads it
- Incremental pipeline reruns
- Experiments tied to Git history

✗ Reach elsewhere

- Billion-row SQL tables → Iceberg, Delta, Hudi
- Online feature serving → a feature store
- Scheduling and retries → Airflow, Dagster, Prefect
- Row-level access control → a governed warehouse

DVC has no scheduler. You run `dvc repro` from an orchestrator: the orchestrator schedules, DVC decides what needs to run and guarantees the lineage.

Lineage and audit

Question	Answered by
Which data trained this model?	<code>dvc.lock</code> at that commit
Which code?	The Git commit
Which parameters and metrics?	<code>params.yaml</code> and <code>metrics.json</code> , in Git



Can we rebuild it byte-for-byte?

Only if the environment was pinned too

That last row is the honest gap. `dvc.lock` pins data and code, not the Python version, the wheels, or the CUDA driver. Close it by hash-pinning a lock file and declaring it, plus the Dockerfile or image digest, in `deps`. **The strongest audit answer is a scheduled rebuild of a past release**, not a diagram.

Erasure versus content addressing

Deleting an object breaks history; keeping it violates the law; rewriting Git history does not touch the bytes.

✓ Design for erasure

- Version derived, aggregated, or pseudonymised data
- Crypto-shredding: encrypt per subject, delete the key
- Partition so deletion is scoped
- Keep raw personal data in a governed system; track its hash

✗ Does not work

- Delete the object and hope
- Rewrite Git history
- Compliance-mode lock plus an erasure obligation
- "We will handle it when asked"

The resolution is always architectural and upstream of the tool: **do not put erasable data in an immutable store.**

The review checklist

Check	Looking for
<code>dvc.lock</code> committed, never hand-edited	The record of what ran
Every script and config in <code>deps</code>	No silent skips
Metrics and plots <code>cache: false</code>	Diffable without a pull



NO credentials in <code>.dvc/config</code>	Nothing in history
Regenerable intermediates <code>push: false</code>	Bounded storage cost
Environment pinned and declared	Reproducibility past code and data
External deps are dated, immutable paths	A source cannot rewrite history
<code>verify true</code> on remotes	Corruption caught, not restored
Bucket versioning enabled	A mistaken delete is survivable
No human credential can delete	The <code>gc --cloud</code> blast radius
CI uses OIDC	Nothing to leak
Fork PRs are read-only	A contributor cannot overwrite data
<code>dvc status</code> gate in CI	The lock cannot drift
Retention is a lifecycle policy	Audited, not clone-triggered
Small-file datasets sharded	Hashing and request costs bounded
No erasable personal data in versioned artifacts	The hard problem, before it is urgent
A scheduled reproduction check exists	Reproducibility monitored, not assumed
Each dataset has a recorded owner	Accountability
Storage cost is visible to the team	Growth gets managed

Common interview questions

01 Explain DVC to an engineer who has never used it.



reproduces a result rather than describing one, and `dvc repro` reruns only what changed.

02 What is the single most dangerous DVC command, and why?

`dvc gc --cloud --workspace`. It deletes every object on the shared remote that is not needed by the currently checked-out revision, which on a repository with twenty branches destroys nineteen branches' data permanently, and with a zero exit code. The mitigations are removing delete permission from human credentials, doing retention as an audited lifecycle policy, and enabling bucket versioning.

03 Why does write access to a data remote matter more than it appears?

Objects live at paths derived from their hash. A client that can write can also `replace` the object at a hash path with different bytes, and every commit referencing that hash then resolves to the wrong data with no error anywhere, because a pointer records what DVC expected, and nothing checks the bytes unless `verify true` is set.

04 How do you give CI credentials without storing a secret?

OIDC or workload identity: the CI provider mints a short-lived token, and the cloud role's trust policy pins the subject to a specific repository and branch. Nothing is stored, so nothing can leak, and a throwaway branch cannot assume the role. For humans, the provider's own SSO, since DVC uses each provider's standard credential chain.

05 What must a fork pull request not be able to do?

Write to the data remote. A fork build runs contributor code with whatever identity the job holds, so a write credential means a contributor can overwrite datasets. Give fork builds a read-only remote or none, and split the workflow so the privileged half (push, promotion, registry updates) never executes fork-supplied code.

06 How do you make a data remote trustworthy?

Bucket versioning so any overwrite or delete is recoverable, object lock so deletion needs a specific privilege, `verify true` so corruption is refused rather than silently restored, encryption at rest, and delete permission removed from every human credential. Retention becomes a declarative lifecycle policy rather than someone running `gc`.



the region, and retrieval from cold classes. The surprise is usually requests on a million-small-file dataset, or egress because CI runs in a different region from the bucket. The largest single saving is almost always `push: false` on intermediates that regenerate in minutes.

08 Where is DVC's scaling wall, and what do you do about it?

Many small files. A million 4 KB objects means a million hash operations, a million transfers, and a `.dir` listing of a million rows. Sharding into tars, Parquet, or WebDataset typically improves `dvc add` by more than tenfold, turns request-bound transfers into throughput-bound ones, and speeds up the data loader. Large files are fine, because hashing is I/O-bound and parallel.

09 Would you use DVC for a billion-row warehouse table?

No. That is a table-format problem: Iceberg, Delta, or Hudi give you time travel and schema evolution natively, and DVC would be duplicating the warehouse. Track the query and its result hash so the pipeline stays reproducible, and let the lakehouse version the table. Insisting DVC handles every data shape is the wrong answer here.

10 Does DVC replace Airflow?

No. It has no scheduler, no retries, and no distributed execution. You run `dvc repro` from an orchestrator: the orchestrator handles scheduling and failure, DVC decides what needs to run and guarantees the lineage. They compose because they solve different problems.

11 A model made a decision affecting a customer eighteen months ago. Prove what trained it.

Find the commit from the model's hash with `git log -S` over `dvc.lock`, check that revision out, `dvc pull`, and confirm `dvc repro --dry` reports up to date. That gives you the data hashes, the code, the parameters, and the metrics. Then be honest about the gap: it does not prove the environment unless the lock file and image digest were also declared as dependencies.

12 What does `dvc.lock` not pin?

The environment. Python version, resolved wheel versions, system libraries, GPU driver. Close it by hash-pinning a dependency lock file and declaring it in `deps`, recording the



13 **How do you prove reproducibility rather than claim it?**

A scheduled job that picks a past release, checks it out on a clean runner, pulls, repros, and asserts the metrics match. It fails on the day an upstream dependency, a deleted object, or an unpinned library breaks reproducibility, rather than on the day someone needs it. That single job is worth more than any lineage documentation.

14 **A customer demands erasure of their records. They are in a versioned dataset. What now?**

Establish the blast radius, which hashes contain the data and which commits and models reference them. Decide what must survive, usually the model and its metrics while the training data goes, and write that down. Rebuild the dataset without the subject, delete the old objects from the remote, every local cache, every CI cache, and any mirror, including old versions in a versioned bucket. Record a tombstone explaining the unresolvable pointer. Then close the class by moving personal data out of the versioned store.

15 **Why are content addressing and erasure in tension?**

An immutable, replicated, hash-addressed store is precisely the wrong shape for "make this specific record disappear everywhere". The same tension exists in Git, in backups, and in any append-only log. The resolution is architectural and upstream of the tool: version derived, aggregated, or pseudonymised data, or encrypt per subject so erasure means destroying a key.

16 **Can you enable compliance-mode object lock and still honour GDPR?**

Not on the same objects. Compliance mode means nobody can delete before expiry, including you, and including under a lawful erasure request. If you are subject to both obligations, keep personal data out of the immutable store or store it encrypted per subject. Decide before enabling it, because compliance mode cannot be walked back.

17 **How do you run a data registry as a product?**

Dated immutable paths rather than `latest.parquet`, a Git tag per release so consumers pin a readable version, a README per dataset with schema, provenance, licence and owner, CODEOWNERS so someone is accountable, a validation stage writing row counts and schema hashes to a `cache: false` metrics file, and a published deprecation policy. Consumers use `dvc import --rev` and `dvc update` deliberately.



nine variants and no canonical source. The fix is turnaround time on requests, not policy. If a reasonable schema change takes three weeks, forking is rational and you will lose. Treat it as a product with a service level or do not call it canonical.

19 How do you run training in CI at scale?

Self-hosted or per-job cloud runners with GPUs, in the same region as the bucket because egress often exceeds compute cost. A warm `.dvc/cache` on protected-branch runners, ephemeral runners for anything triggered by a pull request. OIDC for credentials, and CML comments carrying metrics, parameter, and data diffs plus rendered plots so a reviewer needs to run nothing.

20 Why are self-hosted runners a risk?

They keep state. A persistent runner with a warm cache is fast and is also a shared, writable environment that has executed every contributor's code. Keep warm-cache runners on protected branches, make pull-request runners ephemeral, and never let a fork-triggered job hold the remote's write credential.

21 What would you measure to know your data versioning is healthy?

Bytes and objects per project with the growth trend, the fraction of bytes in unreferenced experiments, the percentage of pipelines where `dvc status` is clean in CI, the age of the oldest successfully reproduced release, the number of remotes with human write access, and median `dvc pull` time in CI. Without numbers, "our data is under control" is an opinion.

22 How would you roll DVC out to six teams?

A template repository: a working pipeline, the CI workflow with OIDC and the `dvc status` gate, the shared `.dvc/config` with the right remote and cache settings. Teams inherit every good decision by cloning, which works and a wiki page does not. Then own the remote layout and its policy centrally, make the sanctioned path the fast path, and publish the metrics.

23 What do you look for reviewing a DVC project?

Not what it does, but what it
guarantees



contain data somebody could lawfully demand you erase.

Final self-test

- Give the whole-series answer in four sentences, then name the most dangerous command.
- Explain why a write credential on a remote is worse than it appears.
- Say how CI gets credentials with nothing stored, and what the trust policy pins.
- Name four controls that make a remote trustworthy.
- List the four cost charges, and the largest single saving.
- Say where DVC's scaling wall is, and the fix plus its three benefits.
- Explain why a warehouse table is not a DVC problem.
- Say what DVC does and does not do relative to an orchestrator.
- Trace a model to its training data in three commands.
- Name what `dvc.lock` does not pin, and how to close the gap.
- Give the metric that proves reproducibility rather than claiming it.
- Explain the tension between erasure and content addressing, and the architectural fix.
- Say why compliance-mode object lock conflicts with an erasure obligation.
- Name five properties that make a data registry a product.
- Give two risks of self-hosted runners and their mitigations.
- Name three health metrics you would publish, and run the review checklist from memory.



PREVIOUS

Senior · Detailed

NEXT UP

Senior · Tips & Tricks





MLOps MENA
COMMUNITY

Free MLOps and AI learning for engineers across the Middle East and North Africa — live sessions, three structured roadmaps, open cohort courses, study groups, and mentorship. We also train, staff, and build for companies across the region.



LEARN

[Roadmaps](#)

[Student Guides](#)

[Courses](#)

[The MLOps Practitioner](#)

[Sessions](#)

[Articles](#)

COMMUNITY

[Meet the team](#)

[Services for companies](#)

[FAQ](#)

[Contact us](#)

[Privacy Policy](#)

WHERE WE ARE

[WhatsApp](#)



MLOps MENA
COMMUNITY

العربية



Join the Community ↗



X

YouTube

GitHub

Zomra



Brainsmingle space

Free to join this week with code **FREECORETEAM**

© 2026 MLOps MENA Community. Free learning for the community.

Built by the community, for the community.